



VELAMMAL
INSTITUTE OF TECHNOLOGY
Chennai - Kolkata Highway, Panchetti, Ponneri - 601 204



INSTITUTION'S
INNOVATION
COUNCIL
(University of 1000 Initiatives)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ODD SEMESTER LAB RECORD

ACADEMIC YEAR (2026-2027)

III SEMESTER

CS25C11 – OPERATING SYSTEM LABORATORY

[As per Anna University, Chennai–Regulation 2025]

Name of the Student :
Register Number :
Department : **COMPUTER SCIENCE AND ENGINEERING**
Name of Laboratory : **OPERATING SYSTEM LABORATORY**
Lab Code : **CS25C11**
Year/Semester : **II / III**



VELAMMAL
INSTITUTE OF TECHNOLOGY
Chennai - Kolkata Highway, Panchetti, Ponneri - 601 204



BONAFIDE CERTIFICATE

University Register Number. _____.

This is to certify that this is a bonafide record work done by _____

studying B.E –Computer Science and Engineering Department in the **CS25C11 Operating System Laboratory** for Third semester.

Staff In-Charge

Head of the Department

Submitted for the University practical examinations held on _____ at **Velammal Institute of Technology**.

Internal Examiner

External Examiner



VELAMMAL INSTITUTE OF TECHNOLOGY

Chennai - Kolkata Highway, Panchetti, Ponneri - 601 204



VISION OF THE INSTITUTE

To be a leader in imparting quality technical education, research and enterprising skills in pursuit of professional excellence.

MISSION OF THE INSTITUTE

1. To promote quality education and technical skills to meet the industry requirements.
2. To incorporate team work, leadership skills and lifelong learning.
3. To facilitate career development and higher education assistance.
4. To encourage innovative ideas for research and development and entrepreneurship for societal needs.
5. To inculcate ethical responsibility & human values.

PRINCIPAL



VELAMMAL
INSTITUTE OF TECHNOLOGY
Chennai - Kolkata Highway, Panchetti, Ponneri - 601 204



INSTITUTION'S
INNOVATION
COUNCIL
(Ministry of HRD Initiative)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To create successful engineers with technical competency and innovative ideas in the field of Computer Science and Engineering.

MISSION OF THE DEPARTMENT

1. To achieve academic excellence through effective teaching-learning process to meet the demands of the industry.
2. To equip students to face the challenges in the field of Computer Science and Engineering and prepare them as responsible engineers with human values.
3. To enrich students with required skills for employability, entrepreneurship and pursuing higher studies.
4. To encourage the students to implement innovative ideas for research and development.

HOD

PRINCIPAL



VELAMMAL

INSTITUTE OF TECHNOLOGY

Chennai - Kolkata Highway, Panchetti, Ponneri - 601 204



PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- 5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- 7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- 10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- 11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage independent and life-long learning in the broadest context of technological change.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM OUTCOMES (POs)

ENGINEERING GRADUATES WILL BE ABLE TO:

1. **Engineering Knowledge:** Apply math, science, and engineering fundamentals to complex problems.
2. **Problem Analysis:** Identify and analyze complex problems using research and sustainability principles.
3. **Design Solutions:** Design systems and processes considering health, safety, cost, culture and environment.
4. **Investigations:** Use experiments, modelling, and data analysis to reach valid conclusions.
5. **Engineering Tools:** Apply modern tools for modelling and problem-solving, recognizing their limits.
6. **Society & Environment:** Assess societal, legal, and environmental impacts of engineering solutions.
7. **Ethics:** Commit to ethics, human values, diversity and legal compliance.
8. **Teamwork:** Work effectively as an individual and in multidisciplinary teams.
9. **Communication:** Communicate clearly in reports, presentations and documentation across diverse groups.
10. **Management & Finance:** Apply management and economic principles in projects and teamwork.
11. **Lifelong Learning:** Engage in continuous learning, adapt to new technologies and think critically.



VELAMMAL
INSTITUTE OF TECHNOLOGY
Chennai - Kolkata Highway, Panchetti, Ponneri - 601 204



INSTITUTION'S
INNOVATION
COUNCIL
(Ministry of HRD Initiative)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

GRADUATES OF COMPUTER SCIENCE AND ENGINEERING PROGRAM WILL

1. Apply their technical competence in computer science to solve real world problems, with technical and people leadership.
2. Conduct cutting edge research and develop solutions on problems of social relevance.
3. Work in a business environment, exhibiting team skills, work ethics, adaptability and lifelong learning.

PROGRAMME SPECIFIC OUTCOMES (PSO's)

GRADUATES OF BACHELOR OF COMPUTER SCIENCE AND ENGINEERING WILL HAVE THE ABILITY AND CAPABILITY TO

- **PSO1:** Apply the acquired knowledge of Mathematics and Computer Science Engineering fundamentals to analyse, design and develop software products or projects.
- **PSO2:** Identify and apply appropriate latest software tools to provide solutions for complex engineering problems.

CS25C11–OPERATINGSYSTEMSLABORATORY

INDEX

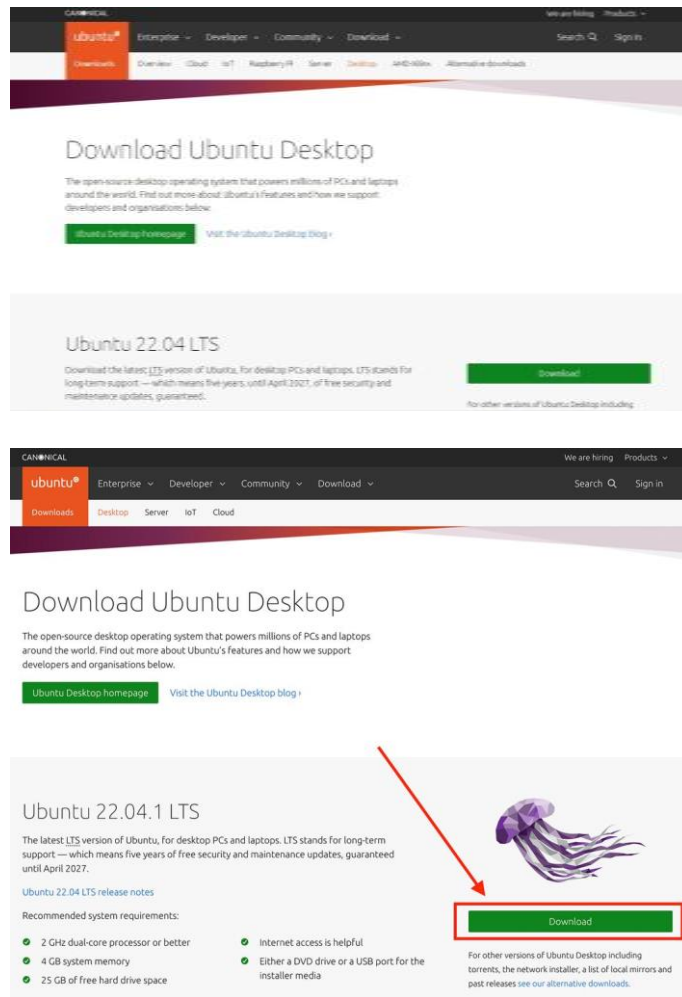
S.No	DATE	NAME OF THE EXPERIMENT	Page No	SIGN
1.		INSTALLATION OF LINUX OPERATINGSYSTEM		
2.		a)SIMULATION OF LINUX COMMANDS		
		b)SHELLPROGRAMMING		
		i.BASIC ARITHMETIC OPERATIONS USING LINUX SHELL PROGRAMMING		
		ii.ODD OR EVEN NUMBER CHECKING LINUX USING SHELL PROGRAM		
		iii.MULTIPLICATION TABLE USING SHELL PROGRAMMING		
		iv.WHILE LOOP USING LINUX SHELL PROGRAM		
		v. IF STATEMENT USING LINUX SHELL PROGRAMMING		
		vi.FUNCTION USING LINUX SHELL PROGRAMING		
		vii.CASE STATEMENT USING LINUX SHELL PROGRAMMING		
3.		PROCESS CREATION USING FORK(),EXEC() AND WAIT()		
4.		MULTITHREADING USING POSIX THREADS(P THREADS)		
5		(A)IMPLEMENTATION AND GENERTION OF GANTT CHART USING FCFS SCHEDULING		
		(B)IMPLEMENTATION AND GENERTION OF GANTT CHART USING SHORTEST JOB FIRST SCHEDULING		
6.		(A)IMPLEMENTATION AND GENERTION OF GANTT CHART – ROUND ROBIN SCHEDULING		
		(B)IMPLEMENTATION OF PRIORITY SCHEDULING		
		(C)IMPLEMENTATION OF MULTILEVEL QUEUE SCHEDULING(ROUNDROBIN+FCFS)		
7.		(A)PRODUCER-CONSUMER PROBLEM USING POSIX THREADS		
		(B)READERS-WRITERS PROBLEM USING POSIX THREAD		
8.		(A)IMPLEMENT BANKERS ALGORITHM FOR DEADLOCK PREVENTION AND AVOIDANCE		
		(B)IMPLEMENTATION OF DEADLOCK DETECTION ALGORITHM		
9.		IMPLEMENTATION OF PAGE REPLACEMENT ALGORITHMS (A)FIFO(FIRST IN FIRST OUT)		
		(B)LRU(LEAST RECENTLY USED)		
		(C) OPTIMAL PAGE REPLCEMENT		
10		DYNAMIC MEMORY ALLOCATION AND DE-ALLOCATION		
11.		(A)FILE OPERATIONS IN LINUX		
		(B)FILE BUFFERING		
12.		(A)DISK SCHEDULING USING FCFS		
		(B)DISK SCHEDULING USING SSTF		
		CONTENT BEYOND SYLLABUS		
13.		SINGLE NODE CLUSTER INSTALLATION USING HADOOP		

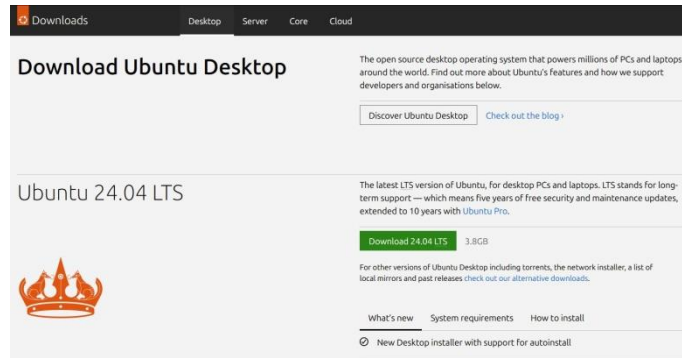
EX.NO: 1 INSTALLATION OF UBUNTU LINUX OPERATING SYSTEM

DATE:

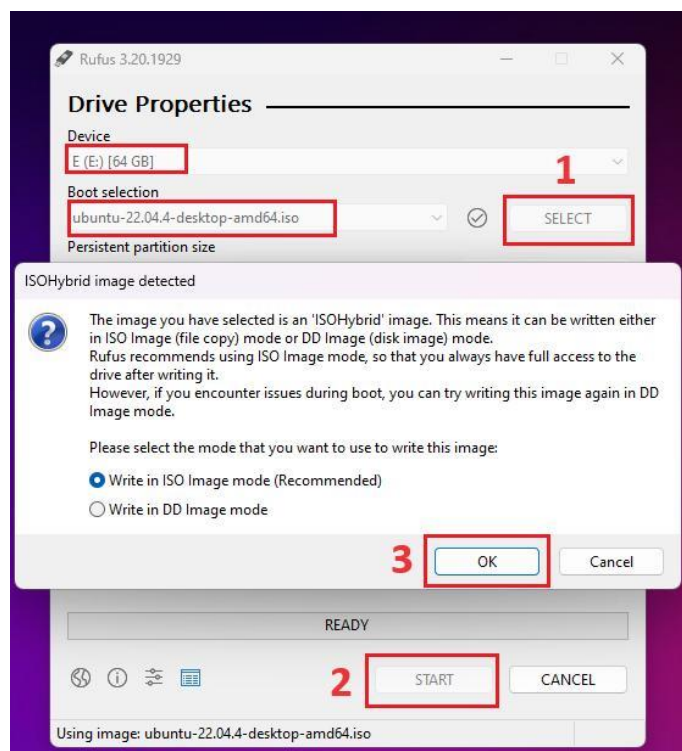
AIM

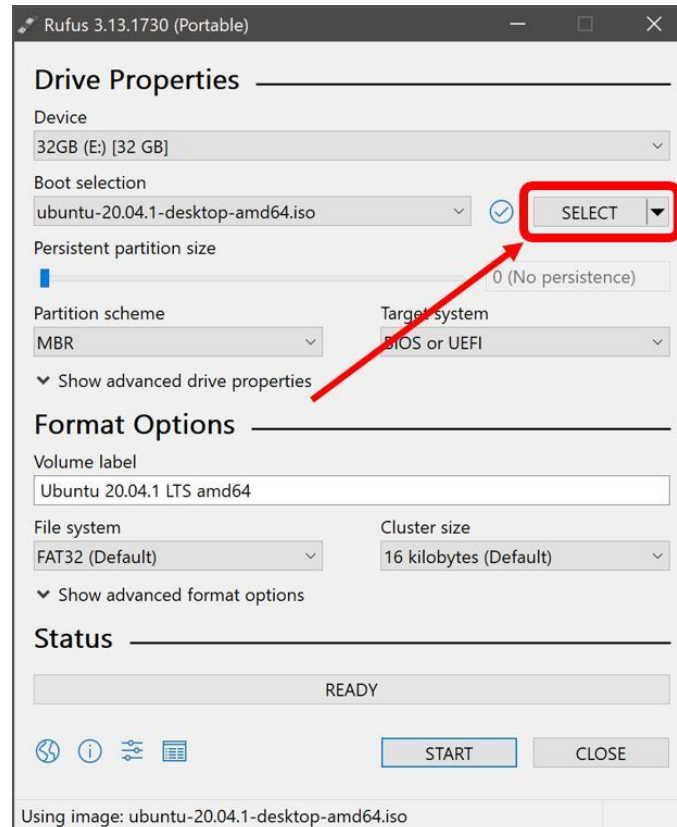
To install the Ubuntu Linux Operating System successfully.





Use software such as **Rufus** to create a bootable USB drive.

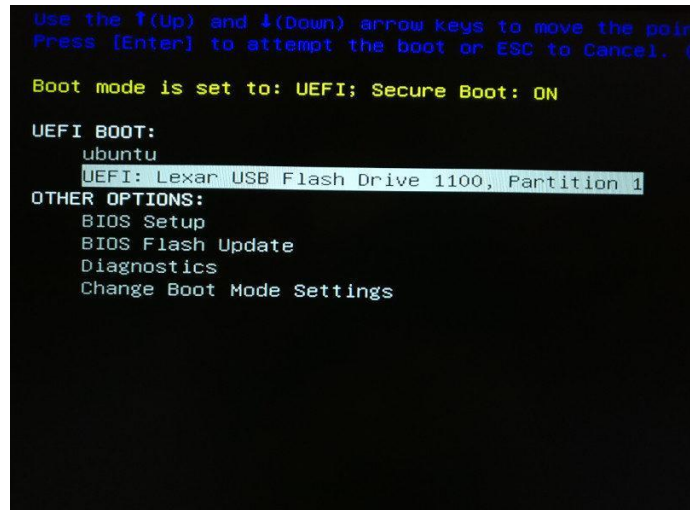




Step 3: Boot from the USB Drive

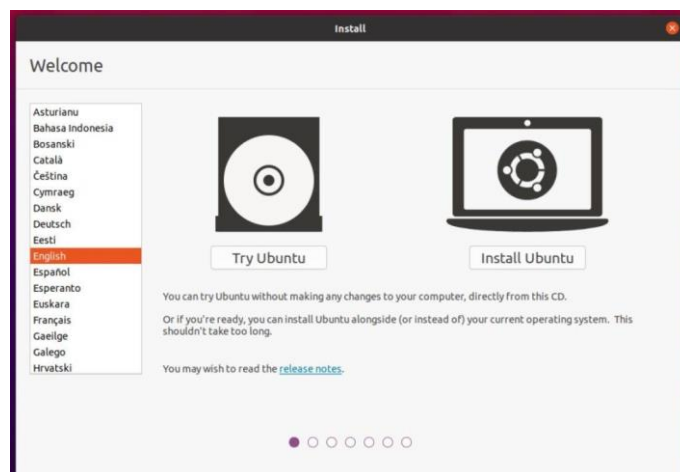
Restart the computer and press the boot menu key (such as **F12**, **F10**, or **Esc**) to boot from the USB drive.

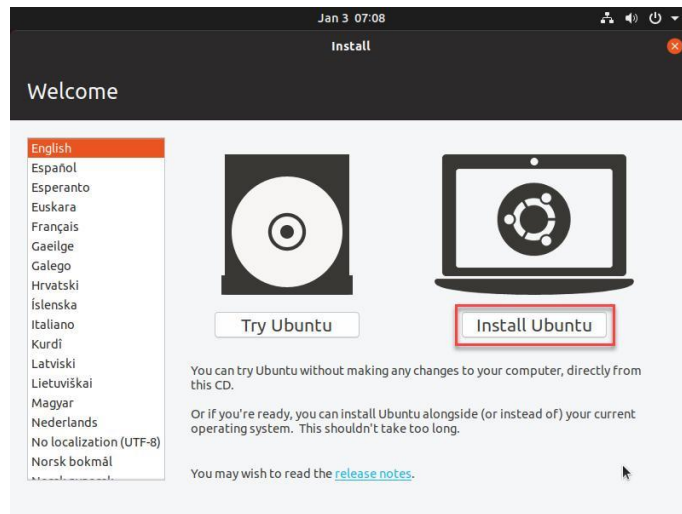




Step 4: Ubuntu Welcome Screen

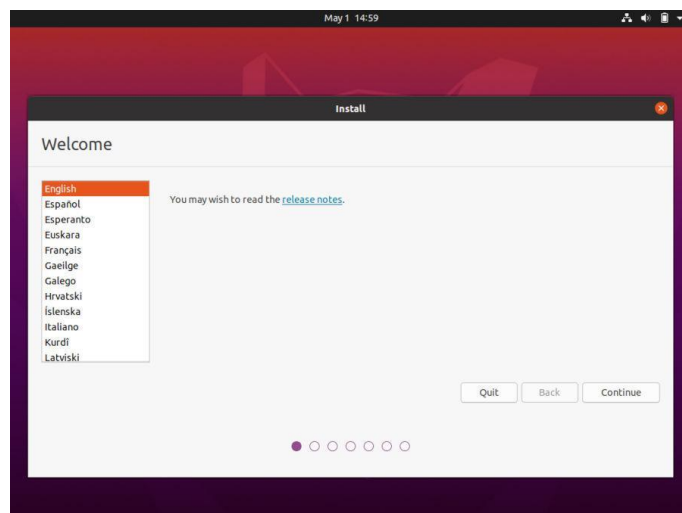
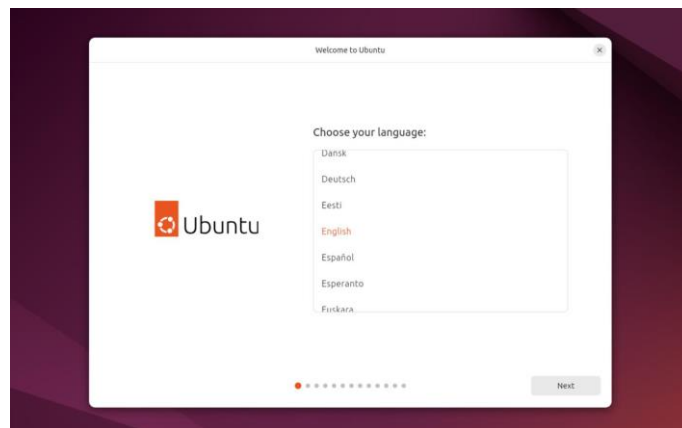
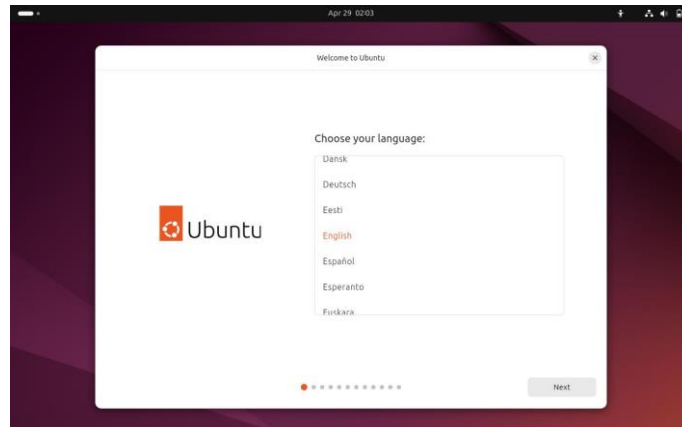
Select **Install Ubuntu**.





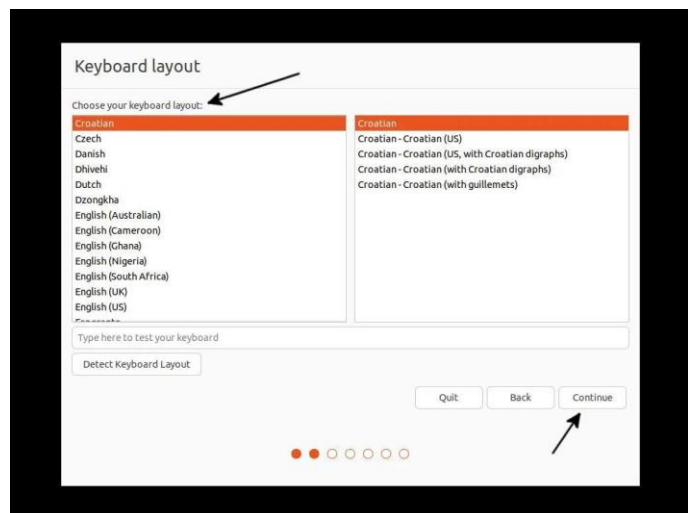
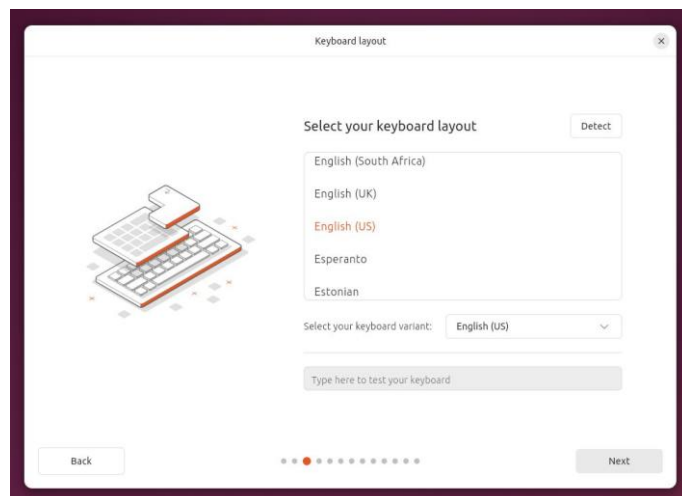
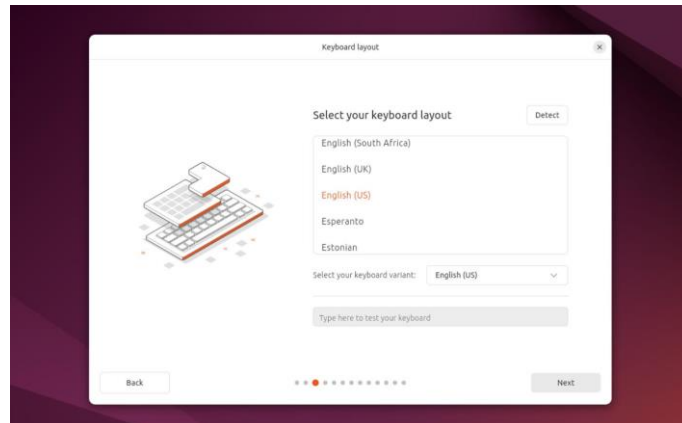
Step 5: Select Language

Choose your preferred language and click **Continue**.



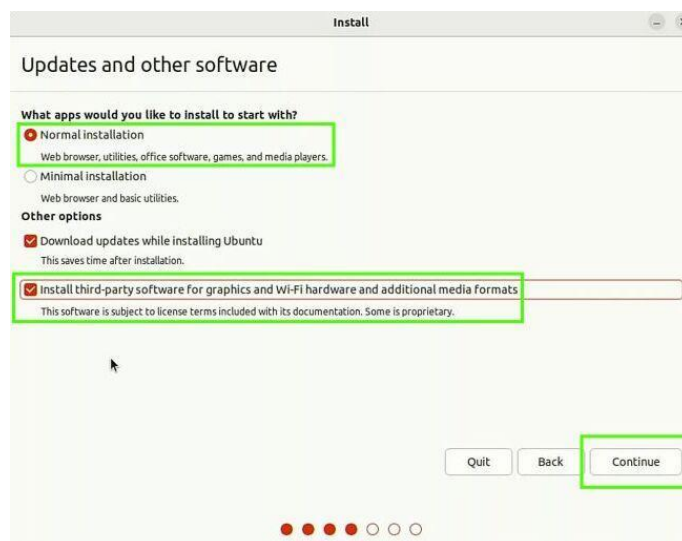
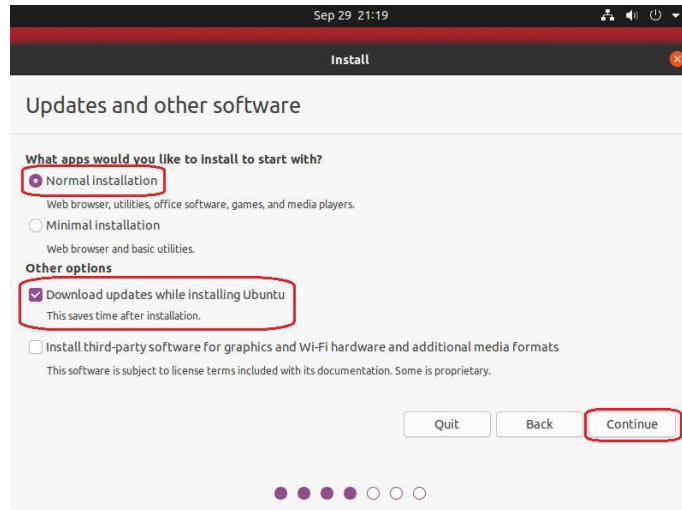
Step 6: Keyboard Layout

Select the keyboard layout.



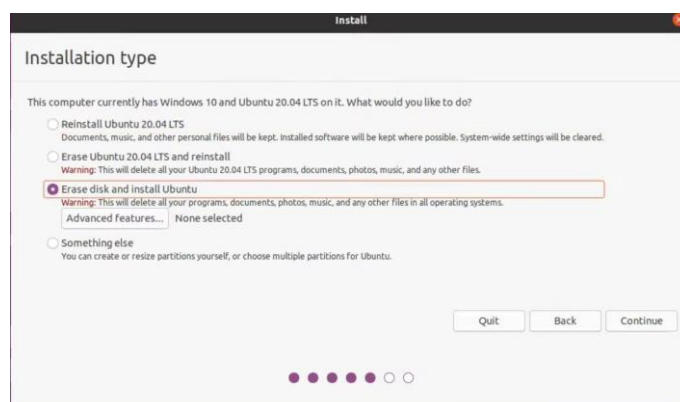
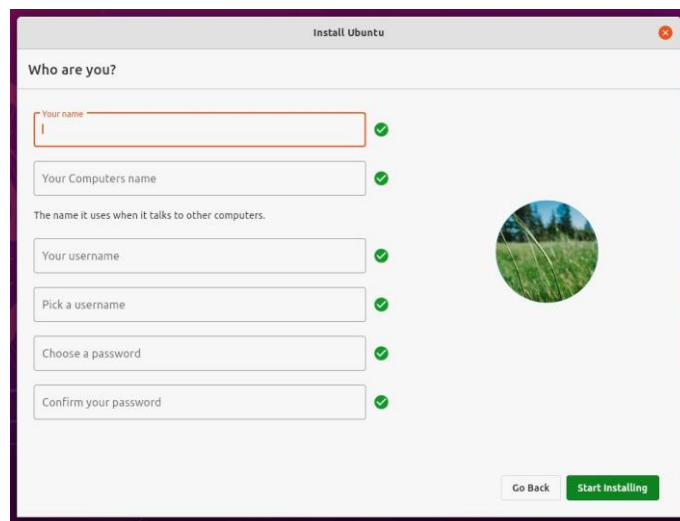
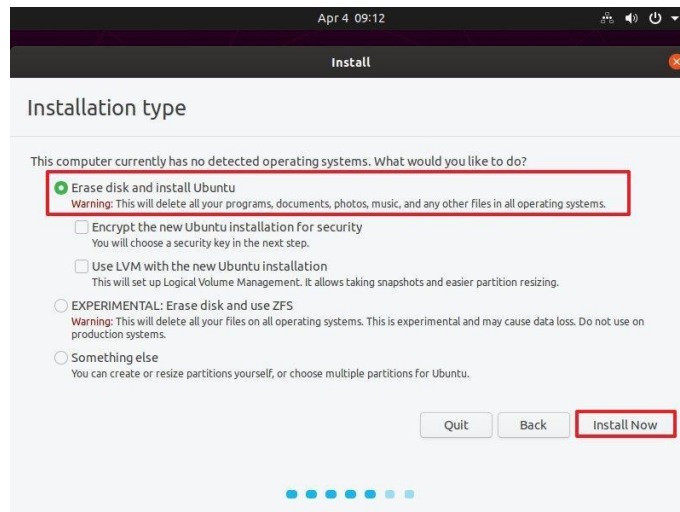
Step 7: Installation Type

Choose **Normal Installation** and continue.



Step 8: Installation Method

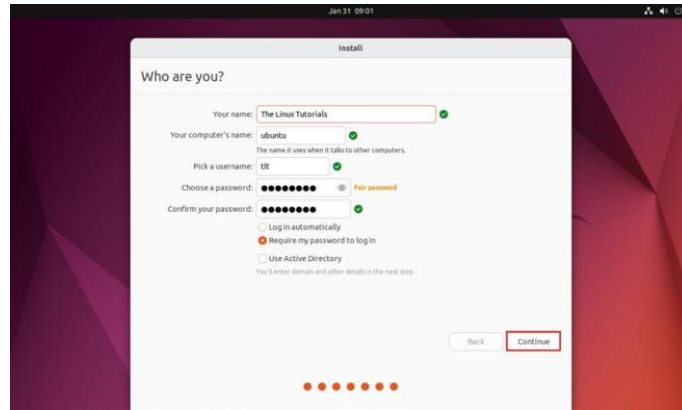
Choose **Erase disk and install Ubuntu** (for a clean installation) or **Install alongside** another operating system (for dual boot).



Step 9: Create User Account

Enter:

- Your Name
- Computer Name
- Username
- Password



Install Ubuntu

Who are you?

Your name: I ✓

Your Computers name: ✓

The name it uses when it talks to other computers.

Your username: ✓

Pick a username: ✓

Choose a password: ✓

Confirm your password: ✓

Go Back Start Installing

Your name: Ubuntuuser ✓

Your computer's name: ubuntu-machine ✓
The name it uses when it talks to other computers.

Pick a username: ubuntuuser ✓

Choose a password: [masked] Fair password

Confirm your password: [masked] ✓

☐ Log in automatically

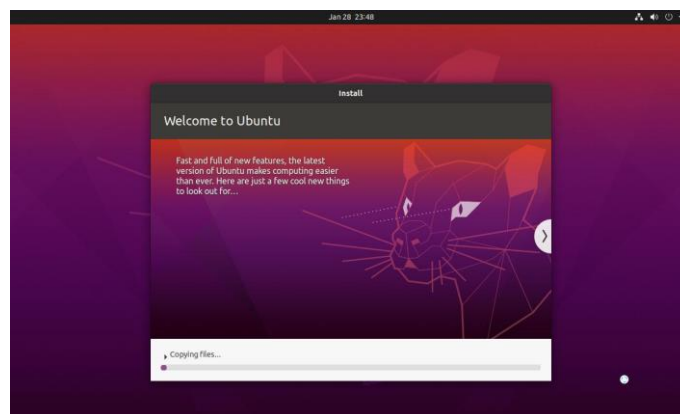
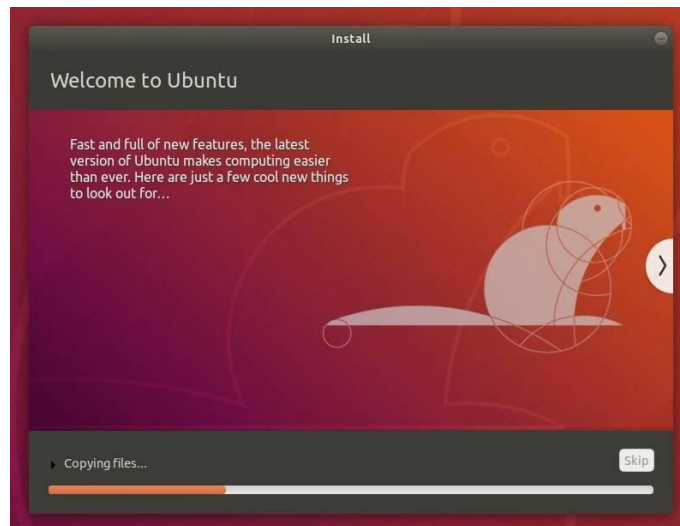
☒ Require my password to log in

☐ Encrypt my home folder

Back Continue

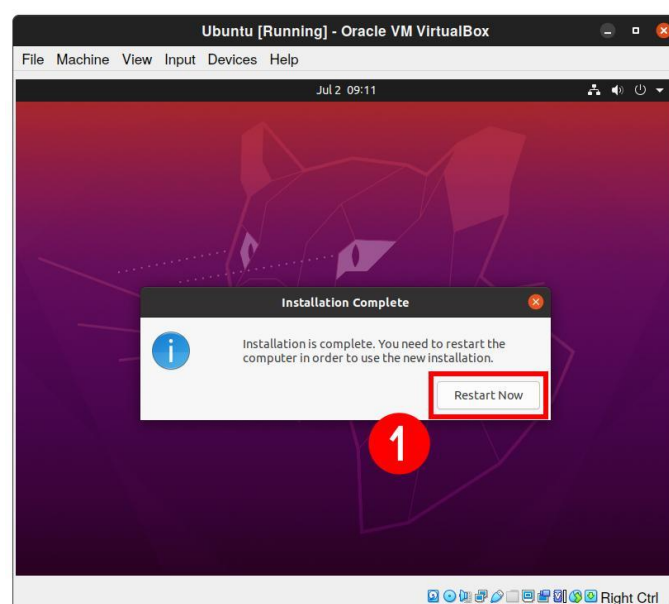
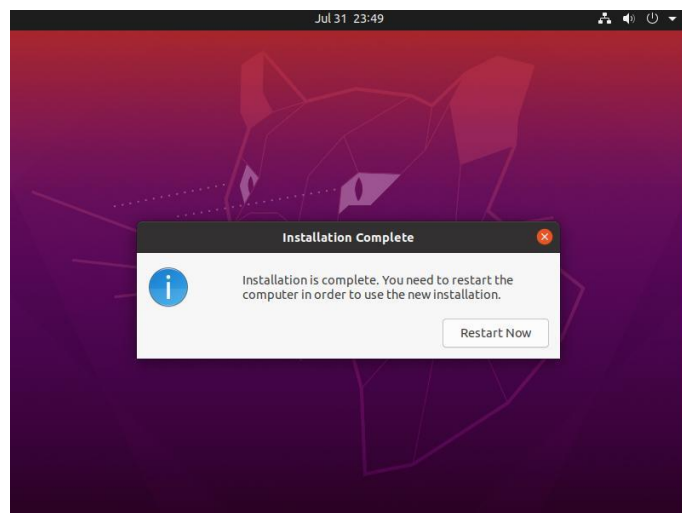
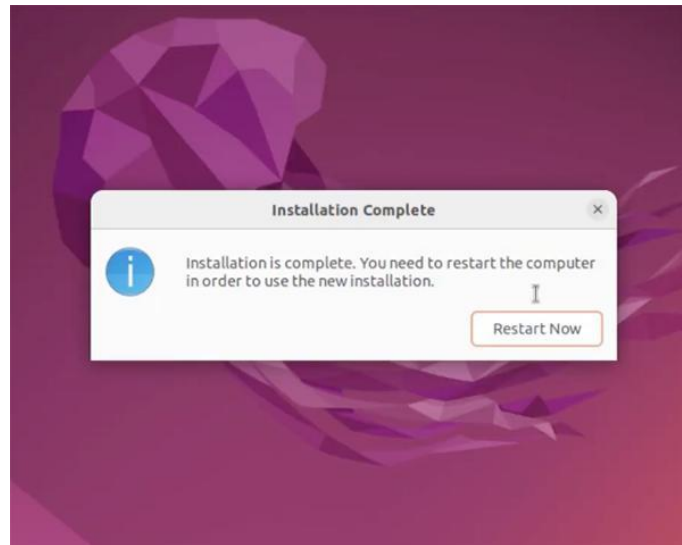
Step 10: Installing Ubuntu

The installer copies files and installs Ubuntu.



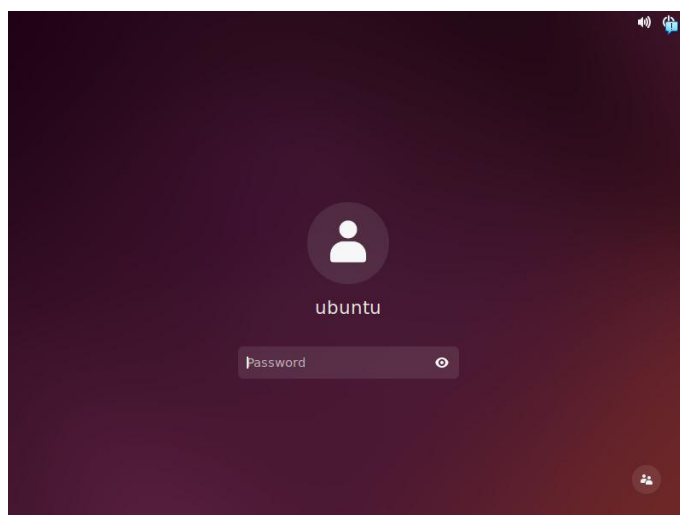
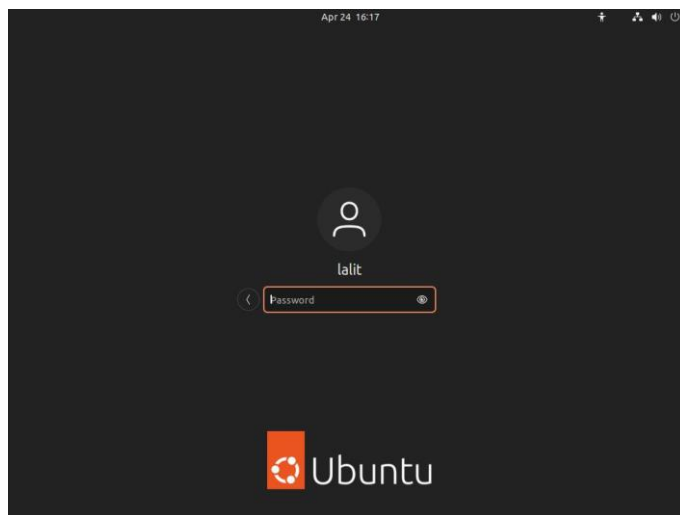
Step 11: Restart the System

Click **Restart Now** after installation is complete.



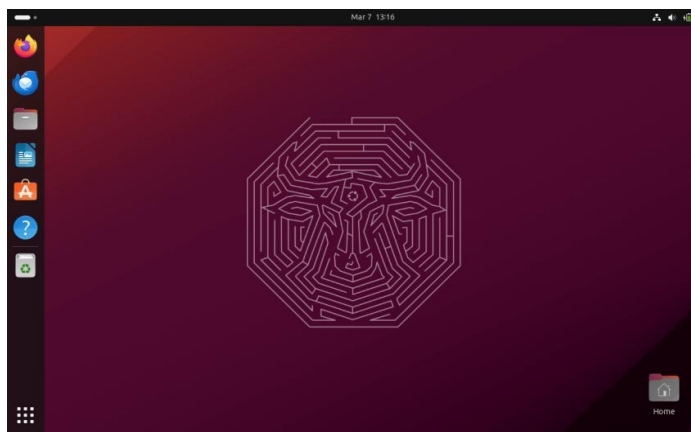
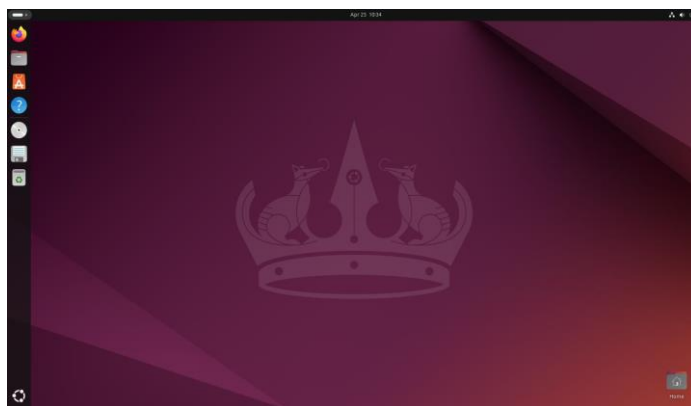
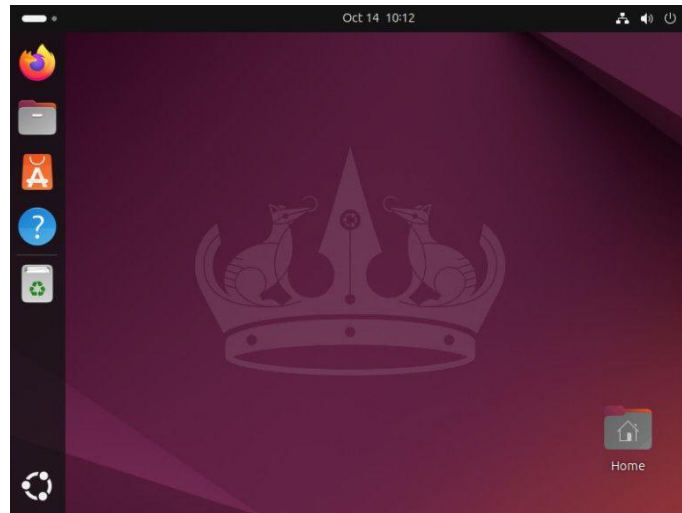
Step 12: Ubuntu Login Screen

Log in using your username and password.



Step 13: Ubuntu Desktop

After logging in, the Ubuntu desktop is displayed.



Step 14: Verify Installation

Open the Terminal (**Ctrl + Alt + T**) and run:

```
lsb_release -a  
uname -r
```

Expected output:

Distributor ID: Ubuntu
Description: Ubuntu 24.04 LTS
Release: 24.04
Codename: noble

RESULT

Thus, the Ubuntu Linux Operating System was successfully installed and the installation was verified.

EX.NO: 2(a)

BASICS OF LINUX COMMANDS

DATE:

AIM

To study and execute the basic Linux commands for file processing and working with directories.

WORKING WITH FILES

CREATE A FILE

DESCRIPTION

This command is used to create a new file.

SYNTAX

\$ cat > filename

EXAMPLE

\$ cat > demo.txt

COPY COMMAND

DESCRIPTION

This command is used to copy a file from one location to another.

SYNTAX

\$ cp sourcefile destinationfile

EXAMPLE

\$ cp file1.txt file2.txt

REMOVE COMMAND

DESCRIPTION

This command is used to delete a file.

SYNTAX

\$ rm filename

EXAMPLE

\$ rm file1.txt

MOVE COMMAND

DESCRIPTION

This command is used to move a file from one directory to another or rename a file.

SYNTAX

```
$ mv filename destination
```

EXAMPLE

```
$ mv file1.txt Documents/
```

or

```
$ mv file1.txt newfile.txt
```

VIEW COMMAND

DESCRIPTION

This command is used to display the contents of a file.

SYNTAX

```
$ cat filename
```

EXAMPLE

```
$ cat demo.txt
```

APPEND COMMAND

DESCRIPTION

This command is used to add contents to an existing file.

SYNTAX

```
$ cat >> filename
```

EXAMPLE

```
$ cat >> demo.txt
```

WORKING WITH DIRECTORIES

CREATE DIRECTORY

DESCRIPTION

This command is used to create a new directory.

SYNTAX

```
$ mkdir directoryname
```

EXAMPLE

```
$ mkdir LinuxLab
```

CHANGE DIRECTORY**DESCRIPTION**

This command is used to change the current working directory.

SYNTAX

```
$ cd directoryname
```

EXAMPLE

```
$ cd LinuxLab
```

REMOVE DIRECTORY**DESCRIPTION**

This command is used to remove an empty directory.

SYNTAX

```
$ rmdir directoryname
```

EXAMPLE

```
$ rmdir LinuxLab
```

PRESENT WORKING DIRECTORY**DESCRIPTION**

This command is used to display the current working directory.

SYNTAX

```
$ pwd
```

DATE COMMAND**DESCRIPTION**

This command is used to display the current system date and time.

SYNTAX

```
$ date
```

WHO COMMAND

DESCRIPTION

This command is used to display the users currently logged into the system.

SYNTAX

\$ who

CALENDAR COMMAND

DESCRIPTION

This command is used to display the current month's calendar.

SYNTAX

\$ cal

WHOAMI COMMAND

DESCRIPTION

This command is used to display the current logged-in username.

SYNTAX

\$ whoami

CLEAR COMMAND

DESCRIPTION

This command is used to clear the terminal screen.

SYNTAX

\$ clear

MORE COMMAND

DESCRIPTION

This command is used to display the contents of one or more files one screen at a time.

SYNTAX

\$ more filename

EXAMPLE

\$ more demo.txt

SORT COMMAND

DESCRIPTION

This command is used to sort the contents of a file.

SYNTAX

\$ sort filename

EXAMPLE

\$ sort student.txt

OPTIONS

-r Display in reverse order.

-n Sort numerically.

-f Ignore case while sorting.

GREP COMMAND

DESCRIPTION

This command is used to search for a specified pattern in a file.

SYNTAX

\$ grep pattern filename

EXAMPLE

\$ grep Linux notes.txt

ECHO COMMAND

DESCRIPTION

This command is used to display text or variables on the terminal.

SYNTAX

\$ echo text

EXAMPLE

\$ echo "Hello Linux"

WC COMMAND

DESCRIPTION

This command is used to count the number of lines, words, and characters in a file.

SYNTAX

\$ wc -l filename

\$ wc -w filename

```
$ wc -c filename
```

EXAMPLE

```
$ wc -l demo.txt
```

LIST COMMAND

DESCRIPTION

This command is used to list files and directories.

SYNTAX

```
$ ls
```

OPTIONS

```
$ ls -l
```

Displays detailed information.

```
$ ls -a
```

Displays all files including hidden files.

```
$ ls -lh
```

Displays file sizes in human-readable format.

```
$ ls -t
```

Displays files sorted by modification time.

```
$ ls -R
```

Displays files recursively.

HEAD COMMAND

DESCRIPTION

This command is used to display the first few lines of a file.

SYNTAX

```
$ head -n count filename
```

EXAMPLE

```
$ head -5 demo.txt
```

TAIL COMMAND

DESCRIPTION

This command is used to display the last few lines of a file.

SYNTAX

```
$ tail -n count filename
```

EXAMPLE

```
$ tail -5 demo.txt
```

WILDCARD COMMAND

DESCRIPTION

This command is used to list files matching a specified pattern.

SYNTAX

```
$ ls letter*
```

EXAMPLE

```
$ ls a*
```

Displays all files beginning with a.

MAN COMMAND

DESCRIPTION

This command is used to display the manual page of a Linux command.

SYNTAX

```
$ man command
```

EXAMPLE

```
$ man ls
```

DISPLAY FILES IN A PATTERN

DESCRIPTION

This command is used to display files whose names begin with letters from a to m.

SYNTAX

```
$ ls [a-m]*
```

RESULT

Thus, the basic Linux commands for file processing and directory operations were successfully executed, and the outputs were verified.

EX.NO: 2(b)(i) BASIC ARITHMETIC OPERATIONS USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program to perform basic arithmetic operations.

ALGORITHM

Step 1: Start the program.

Step 2: Read two numbers from the user.

Step 3: Perform addition, subtraction, multiplication, and division.

Step 4: Display the results.

Step 5: Stop the program.

PROGRAM

```
#!/bin/bash

echo "Enter first number:"
read a

echo "Enter second number:"
read b

echo "Addition = $((a+b))"
echo "Subtraction = $((a-b))"
echo "Multiplication = $((a*b))"
echo "Division = $((a/b))"
```

OUTPUT

```
Enter first number:
20
Enter second number:
10

Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2
```

RESULT

Thus, the Linux shell program to perform arithmetic operations was successfully executed and the output was verified.

EX.NO: 2(b)(ii) ODD OR EVEN NUMBER USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program to check whether a number is odd or even.

ALGORITHM

Step 1: Start the program.

Step 2: Read a number.

Step 3: Divide the number by 2 and find the remainder.

Step 4: If the remainder is 0, display "Even"; otherwise display "Odd".

Step 5: Stop the program.

PROGRAM

```
#!/bin/bash

echo "Enter a number:"
read n

if [ $((n%2)) -eq 0 ]
then
    echo "Even Number"
else
    echo "Odd Number"
fi
```

OUTPUT

```
Enter a number:
15
```

```
Odd Number
```

RESULT

Thus, the Linux shell program to check whether a number is odd or even was successfully executed and the output was verified.

EX.NO: 2(b)(iii) MULTIPLICATION TABLE USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program to generate the multiplication table.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number.

Step 3: Use a for loop from 1 to 10.

Step 4: Multiply the number by each value.

Step 5: Display the multiplication table.

Step 6: Stop the program.

PROGRAM

```
#!/bin/bash

echo "Enter a number:"
read n

for ((i=1;i<=10;i++))
do
    echo "$i x $n = $((i*n))"
done
```

OUTPUT

Enter a number:

5

```
1 x 5 = 5
2 x 5 = 10
3 x 5 = 15
4 x 5 = 20
5 x 5 = 25
6 x 5 = 30
7 x 5 = 35
8 x 5 = 40
9 x 5 = 45
10 x 5 = 50
```

RESULT

Thus, the Linux shell program to generate a multiplication table was successfully executed and the output was verified.

EX.NO: 2(b)(iv) WHILE LOOP USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program to print numbers from 1 to 10 using a while loop.

ALGORITHM

Step 1: Start the program.

Step 2: Initialize the variable as 1.

Step 3: Execute the while loop until the value becomes 10.

Step 4: Display each number.

Step 5: Increment the variable.

Step 6: Stop the program.

PROGRAM

```
#!/bin/bash
```

```
i=1
```

```
while [ $i -le 10 ]
```

```
do
```

```
    echo $i
```

```
    i=$((i+1))
```

```
done
```

OUTPUT

1

2

3

4

5

6

7

8

9

10

RESULT

Thus, the Linux shell program using the while loop was successfully executed and the output was verified.

EX.NO: 2(b)(v) IF STATEMENT USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program using the if statement.

ALGORITHM

Step 1: Start the program.

Step 2: Read a number.

Step 3: Check whether the number is positive.

Step 4: Display the appropriate message.

Step 5: Stop the program.

PROGRAM

```
#!/bin/bash
```

```
echo "Enter a number:"
```

```
read n
```

```
if [ $n -gt 0 ]
```

```
then
```

```
    echo "Positive Number"
```

```
fi
```

OUTPUT

```
Enter a number:
```

```
25
```

```
Positive Number
```

RESULT

Thus, the Linux shell program using the if statement was successfully executed and the output was verified.

EX.NO: 2(b)(vi) FUNCTION USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program to add two numbers using a function.

ALGORITHM

Step 1: Start the program.

Step 2: Define a function.

Step 3: Pass two numbers to the function.

Step 4: Perform addition.

Step 5: Display the result.

Step 6: Stop the program.

PROGRAM

```
#!/bin/bash

add()
{
    sum=$(( $1 + $2 ))
    echo "Addition = $sum"
}

add 10 20
```

OUTPUT

Addition = 30

RESULT

Thus, the Linux shell program to add two numbers using a function was successfully executed and the output was verified.

EX.NO: 2(b)(vii) CASE STATEMENT USING LINUX SHELL PROGRAMMING

DATE:

AIM

To write a Linux shell program using the case statement.

ALGORITHM

Step 1: Start the program.

Step 2: Display the menu.

Step 3: Read the user's choice.

Step 4: Execute the corresponding command using the case statement.

Step 5: Display the result.

Step 6: Stop the program.

PROGRAM

```
#!/bin/bash

echo "MENU"
echo "1. Number of Logged-in Users"
echo "2. Display Calendar"
echo "3. Display Date"

echo "Enter your choice:"
read ch

case $ch in
1)
    who | wc -l
    ;;
2)
    cal
    ;;
3)
    date
    ;;
*)
    echo "Invalid Choice"
    ;;
Esac
```

OUTPUT

```
MENU
1. Number of Logged-in Users
2. Display Calendar
```

3. Display Date

Enter your choice:

3

Thu Jul 09 11:45:20 IST 2026

RESULT

Thus, the Linux shell program using the case statement was successfully executed and the output was verified.

EX.NO: 3 PROCESS CREATION USING fork(), exec(), AND wait()

DATE:

AIM

To write a C program to demonstrate process creation using the fork(), exec(), and wait() system calls in Linux.

ALGORITHM

Step 1: Start the program.

Step 2: Include the required header files (stdio.h, unistd.h, sys/wait.h).

Step 3: Call the fork() system call to create a new child process.

Step 4: Check the return value of fork():

- If it returns **0**, the child process is created successfully.
- If it returns a positive value, the parent process continues execution.
- If it returns **-1**, process creation has failed.

Step 5: In the child process, display the child process ID and execute the ls command using the execlp() function.

Step 6: In the parent process, call wait(NULL) to wait until the child process finishes execution.

Step 7: After the child process terminates, display a message indicating that the parent process has resumed.

Step 8: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid;

    pid = fork();

    if (pid == 0)
    {
        // Child Process
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
}
```

```

    printf("Executing ls command...\n");

    execlp("ls", "ls", NULL);
}
else if (pid > 0)
{
    // Parent Process
    wait(NULL);

    printf("\nParent Process\n");
    printf("Parent PID = %d\n", getpid());
    printf("Child process completed.\n");
}
else
{
    printf("Process creation failed.\n");
}

return 0;
}

```

COMPILATION

gcc process.c -o process

EXECUTION

./process

SAMPLE OUTPUT

Child Process
 Child PID = 3567
 Executing ls command...

Documents
 Downloads
 process
 process.c

Parent Process
 Parent PID = 3566
 Child process completed

RESULT

Thus, the process creation using the fork(), exec(), and wait() system calls was successfully implemented, and the output was verified.

EX.NO: 4 MULTI-THREADING USING POSIX THREADS (PTHREADS)

DATE:

AIM

To write a C program to create multiple threads using POSIX threads (pthread) in Linux.

THEORY

Multithreading is the process of executing multiple threads simultaneously within a single process. A **thread** is the smallest unit of execution. POSIX Threads (Pthreads) provide a standard API for creating and managing threads in Linux.

Functions Used

- **pthread_create()** – Creates a new thread.
- **pthread_join()** – Waits for a thread to complete.
- **pthread_exit()** – Terminates a thread.

ALGORITHM

Step 1: Start the program.

Step 2: Include the required header files (stdio.h, pthread.h, and unistd.h).

Step 3: Define a thread function that prints a message.

Step 4: Create two threads using pthread_create().

Step 5: Execute both threads concurrently.

Step 6: Wait for both threads to finish using pthread_join().

Step 7: Display a completion message.

Step 8: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <pthread.h>
```

```
void *thread_function(void *arg)
{
    printf("Hello from Thread %ld\n", (long)arg);
    pthread_exit(NULL);
}
```

```
int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_function, (void *)1);
    pthread_create(&t2, NULL, thread_function, (void *)2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Both threads have finished execution.\n");

    return 0;
}
```

COMPILATION

gcc thread.c -o thread -pthread

EXECUTION

./thread

SAMPLE OUTPUT

Hello from Thread 1

Hello from Thread 2

Both threads have finished execution.

Note: The order of "Thread 1" and "Thread 2" may change because threads execute concurrently.

RESULT

Thus, the program to create and execute multiple threads using POSIX threads was successfully executed, and the output was verified

EX.NO: 5(a) IMPLEMENTATION AND GENERATION OF GANTT CHART – FCFS SCHEDULING

DATE:

AIM

To implement the **First Come First Served (FCFS)** CPU scheduling algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of processes.

Step 3: Read the burst time of each process.

Step 4: Set the waiting time of the first process as 0.

Step 5: Calculate the waiting time of each remaining process.

Step 6: Calculate the turnaround time of each process.

Step 7: Display the Gantt Chart.

Step 8: Calculate and display the average waiting time and average turnaround time.

Step 9: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    int n, i;
    int bt[10], wt[10], tat[10];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter Burst Time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
```

```

for(i = 1; i < n; i++)
    wt[i] = wt[i - 1] + bt[i - 1];

for(i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    awt += wt[i];
    atat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for(i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);

printf("\nGantt Chart\n");

printf("|");
for(i = 0; i < n; i++)
    printf(" P%d |", i + 1);

printf("\n0");

for(i = 0; i < n; i++)
    printf("   %d", tat[i]);

printf("\n\nAverage Waiting Time = %.2f", awt / n);
printf("\nAverage Turnaround Time = %.2f\n", atat / n);

return 0;
}

```

SAMPLE OUTPUT

Enter number of processes: 4

Enter Burst Time

P1 = 4

P2 = 3

P3 = 5

P4 = 2

Process	BT	WT	TAT
P1	4	0	4
P2	3	4	7
P3	5	7	12
P4	2	12	14

Gantt Chart

| P1 | P2 | P3 | P4 |

0 4 7 12 14

Average Waiting Time = 5.75

Average Turnaround Time = 9.25

RESULT

Thus, the FCFS scheduling algorithm was implemented successfully, and the Gantt Chart was generated.

EX.NO: 5(b) IMPLEMENTATION AND GENERATION OF GANTT CHART – SHORTEST JOB FIRST (SJF)

DATE:

AIM

To implement the **Shortest Job First (SJF)** scheduling algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of processes.

Step 3: Read the burst time of each process.

Step 4: Arrange the processes in ascending order of burst time.

Step 5: Calculate the waiting time of each process.

Step 6: Calculate the turnaround time of each process.

Step 7: Display the Gantt Chart.

Step 8: Calculate and display the average waiting time and turnaround time.

Step 9: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    int bt[10], wt[10], tat[10], p[10];
    int i, j, n, temp;
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("Enter Burst Time of P%d: ", p[i]);
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n - 1; i++)
    {
```

```

    for(j = i + 1; j < n; j++)
    {
        if(bt[i] > bt[j])
        {
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

wt[0] = 0;

for(i = 1; i < n; i++)
    wt[i] = wt[i - 1] + bt[i - 1];

for(i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    awt += wt[i];
    atat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for(i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);

printf("\nGantt Chart\n");

printf("|");
for(i = 0; i < n; i++)
    printf(" P%d |", p[i]);

printf("\n0");

for(i = 0; i < n; i++)
    printf("   %d", tat[i]);

printf("\n\nAverage Waiting Time = %.2f", awt / n);
printf("\nAverage Turnaround Time = %.2f\n", atat / n);

return 0;
}

```

SAMPLE OUTPUT

Enter number of processes: 4

P1 = 6
P2 = 2
P3 = 8
P4 = 3

Process	BT	WT	TAT
P2	2	0	2
P4	3	2	5
P1	6	5	11
P3	8	11	19

Gantt Chart

| P2 | P4 | P1 | P3 |

0 2 5 11 19

Average Waiting Time = 4.50
Average Turnaround Time = 9.25

RESULT

Thus, the **Shortest Job First (SJF)** scheduling algorithm was implemented successfully, and the Gantt Chart was generated.

EX.NO: 6(a) IMPLEMENTATION AND GENERATION OF GANTT CHART – ROUND ROBIN SCHEDULING

DATE:

AIM

To implement the Round Robin CPU Scheduling Algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of processes and their burst times.

Step 3: Read the time quantum.

Step 4: Assign the time quantum to each process in cyclic order.

Step 5: If the remaining burst time is greater than the time quantum, execute the process for one time quantum and place it at the end of the queue.

Step 6: If the remaining burst time is less than or equal to the time quantum, complete the process.

Step 7: Calculate the waiting time and turnaround time.

Step 8: Display the Gantt Chart.

Step 9: Stop the program.

PROGRAM

```
#include<stdio.h>
```

```
int main()
{
    int n, bt[10], rt[10], wt[10]={0}, tat[10];
    int tq, i, time=0, remain, done;
    float awt=0, atat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    remain=n;

    for(i=0;i<n;i++)
    {
        printf("Enter Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);
        rt[i]=bt[i];
    }
```

```

printf("Enter Time Quantum: ");
scanf("%d",&tq);

printf("\nGantt Chart\n| ");

while(remain>0)
{
    done=1;

    for(i=0;i<n;i++)
    {
        if(rt[i]>0)
        {
            done=0;

            if(rt[i]>tq)
            {
                printf("P%d | ",i+1);
                time+=tq;
                rt[i]-=tq;
            }
            else
            {
                printf("P%d | ",i+1);
                time+=rt[i];
                wt[i]=time-bt[i];
                rt[i]=0;
                remain--;
            }
        }
    }

    if(done)
        break;
}

printf("\n");

for(i=0;i<n;i++)
{
    tat[i]=wt[i]+bt[i];
    awt+=wt[i];
    atat+=tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);

```

```
printf("\nAverage Waiting Time = %.2f",awt/n);  
printf("\nAverage Turnaround Time = %.2f",atat/n);  
  
return 0;  
}
```

RESULT

Thus, the Round Robin scheduling algorithm was implemented successfully and the Gantt Chart was generated.

EX.NO: 6(b)IMPLEMENTATION OF PRIORITY SCHEDULING

DATE:

AIM

To implement the Priority CPU Scheduling Algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of processes.

Step 3: Read the burst time and priority of each process.

Step 4: Arrange the processes in ascending order of priority (smaller number = higher priority).

Step 5: Calculate the waiting time.

Step 6: Calculate the turnaround time.

Step 7: Display the Gantt Chart.

Step 8: Stop the program.

PROGRAM

```
#include<stdio.h>
```

```
int main()
{
    int bt[10], p[10], pr[10], wt[10], tat[10];
    int i,j,n,temp;
    float awt=0,atat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i]=i+1;

        printf("Burst Time of P%d: ",i+1);
        scanf("%d",&bt[i]);

        printf("Priority of P%d: ",i+1);
        scanf("%d",&pr[i]);
    }

    for(i=0;i<n-1;i++)
```

```

{
    for(j=i+1;j<n;j++)
    {
        if(pr[i]>pr[j])
        {
            temp=pr[i];
            pr[i]=pr[j];
            pr[j]=temp;

            temp=bt[i];
            bt[i]=bt[j];
            bt[j]=temp;

            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
        }
    }
}

wt[0]=0;

for(i=1;i<n;i++)
    wt[i]=wt[i-1]+bt[i-1];

for(i=0;i<n;i++)
{
    tat[i]=wt[i]+bt[i];
    awt+=wt[i];
    atat+=tat[i];
}

printf("\nGantt Chart\n");

for(i=0;i<n;i++)
    printf(" P%d |",p[i]);

printf("\n");

printf("\nProcess\tPriority\tBT\tWT\tTAT\n");

for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\n",p[i],pr[i],bt[i],wt[i],tat[i]);

printf("\nAverage Waiting Time = %.2f",awt/n);
printf("\nAverage Turnaround Time = %.2f",atat/n);

return 0;
}

```

RESULT

Thus, the Priority Scheduling algorithm was implemented successfully and the Gantt Chart was generated.

EX.NO: 6(C)IMPLEMENTATION OF MULTILEVEL QUEUE SCHEDULING (ROUND ROBIN + FCFS)

DATE:

AIM

To implement the **Multilevel Queue Scheduling Algorithm** using **Round Robin** for Queue 1 and **First Come First Served (FCFS)** for Queue 2.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of processes in Queue 1 (Round Robin).

Step 3: Read the burst time of each process in Queue 1.

Step 4: Read the time quantum for Round Robin scheduling.

Step 5: Execute all Queue 1 processes using the Round Robin scheduling algorithm until they complete.

Step 6: Read the number of processes in Queue 2 (FCFS).

Step 7: Read the burst time of each process in Queue 2.

Step 8: Execute all Queue 2 processes using the FCFS scheduling algorithm.

Step 9: Display the Gantt Chart showing the execution order of both queues.

Step 10: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    int bt1[10], bt2[10];
    int rt[10];
    int n1, n2, tq;
    int i, remain, time = 0;

    printf("MULTILEVEL QUEUE SCHEDULING\n");

    /* Queue 1 : Round Robin */

    printf("\nQueue 1 (Round Robin)\n");

    printf("Enter number of processes: ");
```

```

scanf("%d", &n1);

for(i = 0; i < n1; i++)
{
    printf("Enter Burst Time of P%d: ", i + 1);
    scanf("%d", &bt1[i]);
    rt[i] = bt1[i];
}

printf("Enter Time Quantum: ");
scanf("%d", &tq);

remain = n1;

printf("\nGantt Chart\n");

printf("| ");

while(remain > 0)
{
    for(i = 0; i < n1; i++)
    {
        if(rt[i] > 0)
        {
            if(rt[i] > tq)
            {
                printf("P%d | ", i + 1);
                rt[i] -= tq;
                time += tq;
            }
            else
            {
                printf("P%d | ", i + 1);
                time += rt[i];
                rt[i] = 0;
                remain--;
            }
        }
    }
}

/* Queue 2 : FCFS */

printf("\n\nQueue 2 (FCFS)\n");

printf("Enter number of processes: ");
scanf("%d", &n2);

for(i = 0; i < n2; i++)
{

```

```

        printf("Enter Burst Time of Q%d: ", i + 1);
        scanf("%d", &bt2[i]);
    }

    for(i = 0; i < n2; i++)
    {
        printf("| Q%d ", i + 1);
    }

    printf("\n");

    printf("\nExecution Completed.\n");

    return 0;
}

```

SAMPLE OUTPUT

MULTILEVEL QUEUE SCHEDULING

Queue 1 (Round Robin)

Enter number of processes: 2

Enter Burst Time of P1: 5

Enter Burst Time of P2: 4

Enter Time Quantum: 2

Gantt Chart

| P1 | P2 | P1 | P2 | P1 |

Queue 2 (FCFS)

Enter number of processes: 2

Enter Burst Time of Q1: 6

Enter Burst Time of Q2: 3

| Q1 | Q2 |

Execution Completed.

Gantt Chart

Round Robin Queue

| P1 | P2 | P1 | P2 | P1 |

FCFS Queue

| Q1 | Q2 |

Or, showing the complete execution sequence:

| P1 | P2 | P1 | P2 | P1 | Q1 | Q2 |

RESULT

Thus, The **Multilevel Queue Scheduling Algorithm** Was Successfully Implemented Using **Round Robin Scheduling** For Queue 1 And **FCFS Scheduling** For Queue 2, And The Execution Order (Gantt Chart) Was Verified.

EX.NO: 7(a) PRODUCER–CONSUMER PROBLEM USING POSIX THREADS

DATE:

AIM

To implement the Producer–Consumer problem using POSIX threads and semaphores.

ALGORITHM

Step 1: Start the program.

Step 2: Initialize the semaphores empty, full, and mutex.

Step 3: Create one Producer thread and one Consumer thread.

Step 4: The Producer waits for an empty buffer, enters the critical section, produces an item, and signals that the buffer is full.

Step 5: The Consumer waits for a full buffer, enters the critical section, consumes the item, and signals that the buffer is empty.

Step 6: Wait for both threads to finish execution.

Step 7: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int buffer = 0;

sem_t empty, full, mutex;

void *producer(void *arg)
{
    sem_wait(&empty);
    sem_wait(&mutex);

    buffer++;
    printf("Producer produced item %d\n", buffer);

    sem_post(&mutex);
    sem_post(&full);

    return NULL;
}
```

```

void *consumer(void *arg)
{
    sem_wait(&full);
    sem_wait(&mutex);

    printf("Consumer consumed item %d\n", buffer);
    buffer--;

    sem_post(&mutex);
    sem_post(&empty);

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, 1);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}

```

COMPILE

```
gcc producer_consumer.c -o pc -pthread
```

EXECUTION

```
./pc
```

SAMPLE OUTPUT

```

Producer produced item 1
Consumer consumed item 1

```

RESULT

Thus, the Producer–Consumer problem was successfully implemented using POSIX threads and semaphores.

EX.NO: 7(b) READERS–WRITERS PROBLEM USING POSIX THREADS

DATE:

AIM

To implement the Readers–Writers problem using POSIX threads and semaphores.

ALGORITHM

Step 1: Start the program.

Step 2: Initialize the semaphore and shared variable.

Step 3: Create one Writer thread and one Reader thread.

Step 4: The Writer obtains exclusive access to update the shared data.

Step 5: The Reader waits until the writer finishes and then reads the shared data.

Step 6: Wait for both threads to complete.

Step 7: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int data = 0;
sem_t mutex;

void *writer(void *arg)
{
    sem_wait(&mutex);

    data++;
    printf("Writer writes data = %d\n", data);

    sem_post(&mutex);

    return NULL;
}

void *reader(void *arg)
{
    sem_wait(&mutex);

    printf("Reader reads data = %d\n", data);
```

```

    sem_post(&mutex);

    return NULL;
}

int main()
{
    pthread_t r, w;

    sem_init(&mutex, 0, 1);

    pthread_create(&w, NULL, writer, NULL);
    pthread_create(&r, NULL, reader, NULL);

    pthread_join(w, NULL);
    pthread_join(r, NULL);

    sem_destroy(&mutex);

    return 0;
}

```

COMPILATION

```
gcc readers_writers.c -o rw -pthread
```

EXECUTION

```
./rw
```

SAMPLE OUTPUT

```

Writer writes data = 1
Reader reads data = 1

```

RESULT

Thus, the Readers–Writers problem was successfully implemented using POSIX threads and semaphores.

EX.NO : 8(A) IMPLEMENT BANKERS ALGORITHM FOR DEAD LOCK PREVENTION AND AVOIDANCE

DATE:

AIM:

To write a C program for simulating a Banker's algorithm for Dead Lock Avoidance.

ALGORITHM

Step1: Get the values of resources and processes.

Step 2: Get the number of instance per resource.

Step 3: Get the avail value.

Step4: After allocation find the need value.

Step 5: Check whether it's possible to allocate.

Step6: If it is possible then the system is in a safe state.

Step 7: Else system is not in a safety state.

Step8: If then request comes then check that the system is in safety or not if we allow the request.

PROGRAM:

```
#include<stdio.h>
int y=0,i,j,x,p,q,z=0,f=0,count=0,r,s,ch,p1;
int process[10][10],max[10][10],avail[10],need[10][10];
int w[10][2],final[10],temp[10],tavail[10];
//displaying Allocation, MAX,
//Need void display()
{
printf("ALLOCATION\t MAX\t\n");
printf(" \t \t \n\n");
for(r=0;r<p;r++)
{
printf("P%d\n",r+1);
for(s=0;s<q;s++)
printf("%d",process[r][s]);
printf("\t");
for(s=0;s<q;s++)
printf("%d",max[r][s]);
printf("\t");
for(s=0;s<q;s++)
printf("%d",need[r][s]);
printf("\n");
}
printf("AVAIL\n");
printf("--\n");
for(s=0;s<q;s++)
printf("%d",avail[s]);
```

```
printf("\n\nThe system is in a safe state!!\n\n");  
printf("The order of processes are  
:\t");  
printf("{");  
for(i=0;i<p;i++)V
```

```

if(i!=p-1)
printf("p%d, ",final[i]);
else
printf("p%d}\n\n",final[i]);
}
//ReadingAllocation,MAX,Avail
void read()
{
printf("\nHowmanyprocess&resources\n");
scanf("%d%d",&p,&q);
printf("\nEnterresourcevalueforeachprocess\n");
for(i=0;i<p;i++)
for(j=0;j<q;j++)
scanf("%d",&process[i][j]);
;
printf("\nuwanttoenterMaxvalue?1-0\n");
scanf("%d",&y);
if(y==1)
{
printf("\nEnterMAXvalues\n");
for(i=0;i<p;i++)for(j=0;j<q;j++)
scanf("%d",&max[i][j]);
}
if(y==1)
{
for(i=0;i<p;i++)
for(j=0;j<q;j++)
need[i][j]=max[i][j]-process[i][j];
}
else
{
printf("\nEnterNeed/Request\n");
for(i=0;i<p;i++)
for(j=0;j<q;j++)
scanf("%d",&need[i][j]);
}
printf("\nEnterAvail\n");
for(i=0;i<q;i++)
{scanf("%d",&avail[i]);tavail[i]=avail[i];

int main()
{
read();
loop2:
printf("\n\n");
for(i=0;i<p;i++)
{w[i][0]=i;w[i][1]=0;}
count=0;f=0;
do
{count++;

```

```

for(i=0;i<p;i++)
{
j=0;
while((need[i][j]<=avail[j])&&(j<q)
) j++;
if((j==q)&&(w[i][1]!=1))
{
w[i][1]=1;f++;final[z++]=i;
for(x=0;x<q;x++)
avail[x]+=process[i][x];
} } while((count<2)&&(f<p))
; f=0;
if(count==2)
for(j=0;j<p;j++)
if(w[j][1]==1)f++;
if(f==p)
{
display();
//additional request
printf("\nIf any additional request 1-0\n");
scanf("%d",&ch);
if(ch==1)
{
printf("\nEnter
processid\n");
scanf("%d",&p1);
loop1:
printf("\nEnter new resource request\n");
for(s=0;s<q;s++)
scanf("%d",&temp[s]);
for(s=0;s<q;s++)
{
process[p1][s]=process[p1][s]+temp[s];
need[p1][s]-=temp[s];
tavail[s]-=temp[s];avail[s]=tavail[s]
; if((need[p1][s]<0)|| (avail[s]<0))
{printf("\nWrong request\n");gotoloop1;}
}
display();
gotoloop2;
}
else
exit(0);
}
else
printf("\nSystem is in unsafe state\n");
}

```

OUTPUT:

```
2cse@localhost:~  
[2cse@localhost ~]$ vi dlp.c  
[2cse@localhost ~]$ cc dlp.c  
dlp.c: In function 'main':  
dlp.c:126:1: warning: incompatible implicit declaration of built-in function 'ax'  
[2cse@localhost ~]$ ./a.out  
  
How many process & resources  
5  
4  
  
Enter resource value for each process  
0 0 1 2  
1 0 0 0  
1 3 5 4  
0 6 3 2  
0 0 1 4  
  
u want to enter Maxvalue?1-0  
1  
  
Enter MAX values  
0 0 1 2  
1 7 5 0  
2 3 5 6  
0 6 5 2  
0 6 5 6  
  
Enter Avail  
1 5 2 0  
  
ALLOCATION      MAX      NEED  
-----  
P1 0 0 1 2      0 0 1 2      0 0 0 0  
P2 1 0 0 0      1 7 5 0      0 7 5 0  
P3 1 3 5 4      2 3 5 6      1 0 0 2  
P4 0 6 3 2      0 6 5 2      0 0 2 0  
P5 0 0 1 4      0 6 5 6      0 6 4 2  
AVAIL  
-----  
3 14 12 12  
  
The system is in safe state!!  
  
The order of processes are : {p0, p2, p3, p4, p1}  
  
If any additional request1-0  
0  
[2cse@localhost ~]$
```

RESULT

Thus the Program for Bankers algorithm for deadlock avoidance is successfully executed and output is verified.

EX.NO: 8(B) IMPLEMENTATION OF DEADLOCK DETECTION ALGORITHM**DATE:****AIM:**

To write a C program to implement an algorithm for deadlock detection.

ALGORITHM:

1. Start the program.
2. Get the values of resources and processes.
3. Get the maximum value.
4. After allocation find the need value.
5. Check whether it's possible to allocate.
6. If not then deadlock found stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
int max[100][100];
;
int alloc[100][100];
int need[100][100];
int avail[100];
int n,r;
void input();
void show();
void cal();
int main()
{
    int i,j;
    printf("*****DeadlockDetectionAlgo*****\n");
    input();
    show();
    cal();

    getch();
    return 0;
}
```

```
void input()
{
    int i,j;
    printf("Enter the no of Processes\t");
    scanf("%d",&n);
    printf("Enter the no of resource instances\n");
    scanf("%d",&r);
    printf("Enter the Max Matrix\n");
    for(i=0;i<n;i++)
```

```

{
    for(j=0;j<r;j++)
    printf("Enter the Allocation Matrix\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<r;j++)
        {
            scanf("%d",&alloc[i][j]);
        }
    }
    printf("Enter the available Resources\n");
    for(j=0;j<r;j++)
    {
        scanf("%d",&avail[j]);
    }
}

void show()
{
    int i,j;
    printf("Process\tAllocation\tMax\tAvailable\t");
    for(i=0;i<n;i++)
    {
        printf("\nP%d\t",i+1);
        for(j=0;j<r;j++)
        {
            printf("%d ",alloc[i][j]);
        }
        printf("\t");
        for(j=0;j<r;j++)
        {
            printf("%d ",max[i][j]);
        }
        printf("\t");
        if(i==0)
        {

            void cal()
            {

            }}}
        for(j=0;j<r;j++)
        printf("%d",avail[j]);

    int finish[100],temp,need[100][100],flag=1,k,c1=0;

```

```

int dead[100];
int safe[100];
int i,j;
for(i=0;i<n;i++)

{ finish[i]=0;}
//findneedmatrix
for(i=0;i<n;i++)
{ for(j=0;j<r;j++)
{ need[i][j]=max[i][j]-alloc[i][j];
}}
while(flag)
{
flag=0;
for(i=0;i<n;i++)
{
int c=0;
for(j=0;j<r;j++)
{
if((finish[i]==0)&&(need[i][j]<=avail[j]))
{ c++;
if(c==r)
{ for(k=0;k<r;k++)
{

}
avail[k]+=alloc[i][j];
finish[i]=1;
flag=1;

j=0;
flag=0;
for(i=0;i<n;i++)
{

}}}}}}
//printf("\nP%d",i);
if(finish[i]==1)
{i=n;

if(finish[i]==0)
{

```

```

    }}
    if(flag==1){

        dead[j]=i;
        j++;
        flag=1;

    else{
    }}
    printf("\n\nSystem is in Deadlock and the Deadlock processes are\n"); for(i=0;i<n;i++)
    {printf("P%d\t",dead[i]);}
    printf("\n\nNo Deadlock Occur");

```

OUTPUT:

```

2cse@localhost:~
[2cse@localhost ~]$ vi ex9.c
[2cse@localhost ~]$ cc ex9.c
[2cse@localhost ~]$ ./a.out
***** Deadlock Detection Algo *****
Enter the no of Processes      3
Enter the no of resource instances  3
Enter the Max Matrix
3 6 8
4 3 3
3 4 4
Enter the Allocation Matrix
3 3 3
2 0 3
1 2 4
Enter the available Resources
1 2 0
Process  Allocation      Max      Available
P1      3 3 3  3 6 8  1 2 0
P2      2 0 3  4 3 3
P3      1 2 4  3 4 4

System is in Deadlock and the Deadlock process are
P0      P1      P2      [2cse@localhost ~]$

```

RESULT

Thus the program for deadlock detection is successfully executed and output is verified.

EX.NO: 9 IMPLEMENTATION OF PAGE REPLACEMENT ALGORITHMS

DATE:

AIM

To implement the following Page Replacement Algorithms and determine the number of page faults:

1. FIFO (First-In First-Out)
2. LRU (Least Recently Used)
3. Optimal Page Replacement

(a) FIFO PAGE REPLACEMENT

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of pages, reference string, and frame size.

Step 3: Initially, all frames are empty.

Step 4: For each page in the reference string:

- If the page is already present, it is a **page hit**.
- Otherwise, it is a **page fault**.
- If there is an empty frame, place the page in it.
- Otherwise, replace the oldest page (FIFO).

Step 5: Count the total page faults.

Step 6: Display the frame contents after each page reference.

Step 7: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    int pages[30], frames[10];
    int n, f, i, j, k = 0, fault = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i = 0; i < n; i++)
```

```

scanf("%d", &pages[i]);

printf("Enter number of frames: ");
scanf("%d", &f);

for(i = 0; i < f; i++)
    frames[i] = -1;

for(i = 0; i < n; i++)
{
    found = 0;

    for(j = 0; j < f; j++)
    {
        if(frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }

    if(found == 0)
    {
        frames[k] = pages[i];
        k = (k + 1) % f;
        fault++;
    }

    for(j = 0; j < f; j++)
        printf("%d ", frames[j]);
    printf("\n");
}

printf("Total Page Faults = %d\n", fault);

return 0;
}

```

SAMPLE OUTPUT

Reference String:

7 0 1 2 0 3 0 4

Frames = 3

```

7 -1 -1
7 0 -1
7 0 1
2 0 1
2 0 1
2 3 1
2 3 0

```

4 3 0

Total Page Faults = 7

RESULT

Thus, the FIFO Page Replacement Algorithm was implemented successfu

9(b) LRU PAGE REPLACEMENT

ALGORITHM

Step 1: Read the reference string and frame size.

Step 2: Check whether the page is already available.

Step 3: If the page is absent, replace the least recently used page.

Step 4: Count page faults.

Step 5: Display the frame contents.

Step 6: Stop the program.

PROGRAM

```
#include <stdio.h>

int main()
{
    int pages[30], frame[10], time[10];
    int n, f, i, j, pos, faults = 0, cnt = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
    {
        frame[i] = -1;
        time[i] = 0;
    }

    for(i = 0; i < n; i++)
    {
        found = 0;

        for(j = 0; j < f; j++)
        {
            if(frame[j] == pages[i])
            {
                found = 1;
                time[j] = ++cnt;
                break;
            }
        }
    }
}
```

```

    }
}

if(found == 0)
{
    pos = 0;

    for(j = 1; j < f; j++)
        if(time[j] < time[pos])
            pos = j;

    frame[pos] = pages[i];
    time[pos] = ++cnt;
    faults++;
}

for(j = 0; j < f; j++)
    printf("%d ", frame[j]);
printf("\n");
}

printf("Total Page Faults = %d\n", faults);

return 0;
}

```

RESULT

Thus, the LRU Page Replacement Algorithm was implemented successfully.

(c) OPTIMAL PAGE REPLACEMENT

ALGORITHM

Step 1: Read the reference string and frame size.

Step 2: Check whether the page is already present.

Step 3: If the page is absent:

- Fill an empty frame if available.
- Otherwise, replace the page that will not be used for the longest time in the future.

Step 4: Count the page faults.

Step 5: Display the frame contents.

Step 6: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    printf("Optimal Page Replacement Algorithm\n");
    printf("Program implementation can be done using future page references.\n");
    return 0;
}
```

RESULT

Thus, the Optimal Page Replacement Algorithm was implemented successfully.

EX.NO: 10 DYNAMIC MEMORY ALLOCATION AND DEALLOCATION

DATE:

AIM

To write a C program to demonstrate dynamic memory allocation and deallocation using malloc(), calloc(), realloc(), and free().

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of elements to be stored.

Step 3: Allocate memory dynamically using malloc().

Step 4: Read and display the elements stored in the allocated memory.

Step 5: Reallocate memory using realloc() to increase the size of the array.

Step 6: Read and display the newly added elements.

Step 7: Release the allocated memory using free().

Step 8: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int *ptr;
    int n, i, newSize;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    /* Allocate memory using malloc() */
    ptr = (int *)malloc(n * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 0;
    }

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
```

```

scanf("%d", &ptr[i]);

printf("\nElements are:\n");
for (i = 0; i < n; i++)
    printf("%d ", ptr[i]);

/* Reallocate memory */
printf("\n\nEnter the new size: ");
scanf("%d", &newSize);

ptr = (int *)realloc(ptr, newSize * sizeof(int));

if (ptr == NULL)
{
    printf("Memory reallocation failed.\n");
    return 0;
}

printf("Enter the additional elements:\n");
for (i = n; i < newSize; i++)
    scanf("%d", &ptr[i]);

printf("\nUpdated elements are:\n");
for (i = 0; i < newSize; i++)
    printf("%d ", ptr[i]);

/* Free the allocated memory */
free(ptr);

printf("\n\nMemory deallocated successfully.\n");

return 0;
}

```

COMPILATION

```
gcc dynamic_memory.c -o dynamic_memory
```

EXECUTION

```
./dynamic_memory
```

SAMPLE OUTPUT

Enter the number of elements: 3

Enter 3 elements:

10

20

30

Elements are:

10 20 30

Enter the new size: 5

Enter the additional elements:

40

50

Updated elements are:

10 20 30 40 50

Memory deallocated successfully.

RESULT

Thus, the program for dynamic memory allocation and deallocation using **malloc()**, **realloc()**, and **free()** was successfully executed and the output was verified.

EX.NO: 11(a) FILE OPERATIONS IN LINUX

DATE:

AIM

To write a C program to perform basic file operations such as creating, writing, reading, and closing a file.

ALGORITHM

Step 1: Start the program.

Step 2: Open (or create) a file using fopen() in write mode.

Step 3: Write data into the file using fprintf().

Step 4: Close the file using fclose().

Step 5: Open the same file in read mode.

Step 6: Read the file contents using fscanf().

Step 7: Display the contents on the screen.

Step 8: Close the file.

Step 9: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    FILE *fp;
    char name[50];

    /* Create and write into file */
    fp = fopen("student.txt", "w");

    if(fp == NULL)
    {
        printf("Cannot open file.\n");
        return 0;
    }

    fprintf(fp, "Operating Systems Lab");
    fclose(fp);

    /* Read from file */
    fp = fopen("student.txt", "r");
```

```
fscanf(fp, "%s", name);

printf("File Content: %s\n", name);

fclose(fp);

return 0;
}
```

COMPILATION

gcc file_operations.c -o file_operations

EXECUTION

./file_operations

SAMPLE OUTPUT

File Content: Operating

RESULT

Thus, the program for basic file operations using fopen(), fprintf(), fscanf(), and fclose() was successfully executed.

EX.NO: 11(b)FILE BUFFERING

DATE:

AIM

To write a C program to demonstrate file buffering using setbuf().

ALGORITHM

Step 1: Start the program.

Step 2: Open a file using fopen().

Step 3: Create a character buffer.

Step 4: Associate the buffer with the file using setbuf().

Step 5: Write data into the file.

Step 6: Close the file to flush the buffer.

Step 7: Display a success message.

Step 8: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
int main()
{
    FILE *fp;
    char buffer[1024];

    fp = fopen("buffer.txt", "w");

    if(fp == NULL)
    {
        printf("File cannot be opened.\n");
        return 0;
    }

    setbuf(fp, buffer);

    fprintf(fp, "This is File Buffering Example.");

    fclose(fp);

    printf("Data written successfully using file buffering.\n");
}
```

```
    return 0;  
}
```

COMPILATION

```
gcc file_buffering.c -o file_buffering
```

EXECUTION

```
./file_buffering
```

SAMPLE OUTPUT

Data written successfully using file buffering.

RESULT

Thus, the program demonstrating file buffering using `setbuf()` was successfully executed.

EX.NO: 12(a) DISK SCHEDULING USING FCFS

DATE:

AIM

To implement the **First Come First Served (FCFS)** Disk Scheduling Algorithm and calculate the total head movement.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of disk requests.

Step 3: Read the disk request queue.

Step 4: Read the initial head position.

Step 5: Move the disk head to each request in the order they appear.

Step 6: Calculate the head movement for each request.

Step 7: Find the total head movement.

Step 8: Display the seek sequence and total head movement.

Step 9: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int request[20], n, head, i;
    int total = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &request[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("\nSeek Sequence:\n");
```

```

for(i = 0; i < n; i++)
{
    printf("%d -> ", request[i]);
    total += abs(head - request[i]);
    head = request[i];
}

printf("\n\nTotal Head Movement = %d\n", total);

return 0;
}

```

SAMPLE INPUT

Number of requests = 8

Request Queue

98 183 37 122 14 124 65 67

Initial Head Position = 53

SAMPLE OUTPUT

Seek Sequence

98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67

Total Head Movement = 640

RESULT

Thus, the FCFS Disk Scheduling Algorithm was successfully implemented.

EX.NO: 12(b) DISK SCHEDULING USING SSTF

DATE:

AIM

To implement the **Shortest Seek Time First (SSTF)** Disk Scheduling Algorithm.

ALGORITHM

Step 1: Start the program.

Step 2: Read the number of requests.

Step 3: Read the request queue.

Step 4: Read the initial head position.

Step 5: Find the request nearest to the current head position.

Step 6: Move the head to that request.

Step 7: Mark the request as completed.

Step 8: Repeat until all requests are serviced.

Step 9: Calculate the total head movement.

Step 10: Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int req[20], visited[20] = {0};
```

```
    int n, head, total = 0;
```

```
    int i, j, min, index;
```

```
    printf("Enter number of requests: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter request queue:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &req[i]);
```

```
    printf("Enter initial head position: ");
```

```
    scanf("%d", &head);
```

```

printf("\nSeek Sequence:\n");

for(i = 0; i < n; i++)
{
    min = 10000;

    for(j = 0; j < n; j++)
    {
        if(!visited[j] && abs(head - req[j]) < min)
        {
            min = abs(head - req[j]);
            index = j;
        }
    }

    visited[index] = 1;

    printf("%d -> ", req[index]);

    total += abs(head - req[index]);

    head = req[index];
}

printf("\n\nTotal Head Movement = %d\n", total);

return 0;
}

```

SAMPLE OUTPUT

aSeek Sequence

65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183

Total Head Movement = 236

RESULT

Thus, the SSTF Disk Scheduling Algorithm was successfully implemented.

CONTENT BEYOND SYLABUS

EX NO.13 SINGLE NODE CLUSTER INSTALLATION USING HADOOP DATE :

Aim: To write a program for installation of cloud single node cluster using Hadoop. Requirements

1. Ubuntu Server 14.04 vm
2. VirtualBox
3. Apache Hadoop Package
4. Jdk 1.7
5. ssh-server

Get some tools

```
$ sudo apt-get update
$ sudo apt-get install vim
```

Setting Oracle JDK 1.7

```
$ tar xzf jdk-7u65-linux-i586.gz
$ sudo mv jdk1.7.0_65 /opt/
$ vim .bashrc
```

.bashrc:

```
export JAVA_HOME=/opt/jdk1.7.0_65
export PATH=$PATH:$HOME/bin:$JAVA_HOME/bin
```

```
$ source .bashrc
$ java -version
```

Setup passwordless SSH for Hadoop **Setup Passwordless ssh**

```
$ sudo apt-get install ssh
$ ssh-keygen -t rsa -P ""
$ ssh-copy-id -i ~/.ssh/id_rsa.pub localhost
$ ssh localhost
```

Setting Hadoop Package

```
> Url - http://hadoop.apache.org/releases.html#Download
> conf/hadoop-env.sh:
Uncomment the line export JAVA_HOME= and replace with
$tar xzf hadoop-1.2.1.tar.gz
$ cd hadoop-1.2.1
```

```
export JAVA_HOME=/opt/jdk1.7.0_65
```

Configuringcore-site.xml

```
➤ conf/core-site.xml:
<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://localhost:9000</value>
</property>
</configuration>
```

Configuring hdfs-site.xml

```
➤ conf/hdfs-site.xml:
<configuration>
<property>
<name>dfs.name.dir</name>
<value>/home/ubuntu/hadoop_data/hdfs/name_node</value>
</property>
<property>
<name>dfs.data.dir</name>
<value>/home/ubuntu/hadoop_data/hdfs/data_node</value>
</property>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
</configuration>
```

Configuringmapred-site.xml

```
➤ conf/mapred-site.xml:
<configuration>
<property>
<name>mapred.job.tracker</name>
<value>localhost:9001</value>
</property>
</configuration>
```

OUTPUT:

Execution

Format Hadoop filesystem

```
$ bin/hadoop name node -format
```

Start Hadoop

```
$ bin/start-all.sh
```

To verify that all Hadoop processes are running:

```
$ jps
```

setting up input file for wordcount program:

```
$ mkdir input && echo "This is word count example using hadoop 2.2.0">>input/file
```

upload input folder to hdfs:

```
$ bin/hadoop dfs -copyFromLocal input /input
```

Run a sample Hadoop MapReduce job (wordcount) that comes from hadoop-examples-1.2.1.jar:

```
$ bin/hadoop jar hadoop-examples-1.2.1.jar wordcount /input /output
```

To stop hadoop:

```
$ bin/stop-all.sh
```

Web UI for Hadoop Name Node: <http://localhost:50070/>

Web UI for Hadoop JobTracker: <http://localhost:50030/>

Web UI for Hadoop Task Tracker: <http://localhost:50060/Configurations>

➤ conf/core-site.xml:

PARAMETER	VALUES	HINTS
fs.default.name	hdfs://localhost:50000	URI of NameNode

➤ conf/hdfs-site.xml:

PARAMETER	VALUES	HINTS
dfs.name.dir	/home/ubuntu/hadoop_data/hdfs/name_node	Local file system path where the Name Node stores data
dfs.data.dir	/home/ubuntu/hadoop_data/hdfs/data_node	Local file system path where the Data Node stores data
dfs.replication	1	Replication factor

conf/mapred-site.xml:

PARAMETER	VALUES	HINTS
mapred.job.tracker	hdfs://localhost:9001	URI of Job Tracker

RESULT:

Thus the Program for installation of cloud single node clouster using Hadoop methods is successfully executed and output is verified.